Project Structure

```
src/main/java/com/smarthome/
├── model/               Data models (POJOs)
├── service/             Business logic services
├── util/                Utility classes
└── SmartHomeDashboard.java  Main application class

src/test/java/com/smarthome/     Unit and integration tests
```

OOP Concepts Implemented

1. Encapsulation

Definition: Bundling data and methods that operate on that data within a single unit, with controlled access.

Implementation Examples:

Customer Class (`Customer.java:10-260`)

java

```
public class Customer {
    private String email;                    // Private fields
    private String fullName;
    private String password;
    private List<Gadget> gadgets;

    public String getEmail() {               // Public getter methods
        return email;
    }

    public void setEmail(String email) {     // Public setter methods
        this.email = email;
    }
}
```

Gadget Class (`Gadget.java:13-24`)

java

```
public class Gadget {
    private String type;                     // Encapsulated fields
    private String model;
    private String roomName;
    private String status;
    private double powerRatingWatts;

    private void updateUsageAndEnergy() {    // Private helper method
        // Internal logic encapsulated
    }
}
```

Benefits Achieved:

- Data protection through private fields
- Controlled access via getter/setter methods
- Internal logic hidden from external classes

2. Constructor Overloading

Definition: Multiple constructors with different parameter sets for flexible object creation.

Implementation.

Customer Class (`Customer.java:21-43`)

java

```
public Customer() {                          // Default constructor
    this.gadgets = new ArrayList<>();
    this.groupMembers = new ArrayList<>();
    // ... initialization
}

public Customer(String email, String fullName, String password) {  // Parameterized constructor
```

```
    this.email = email;
    this.fullName = fullName;
    this.password = password;
    // ... initialization
}
```

Gadget Class (`Gadget.java:25-41`)
java

```
public Gadget() {                           // Default constructor
    this.status = GadgetStatus.OFF.name();
    // ... default initialization
}

public Gadget(String type, String model, String roomName) {  // Parameterized constructor
    this.type = type;
    this.model = model;
    this.roomName = roomName;
    // ... initialization
}
```

3. Method Overloading
Definition: Multiple methods with the same name but different parameters.
Implementation Examples:

Customer Class (`Customer.java:154-159`)
java

```
public boolean isGroupAdmin() {              // No parameters
    return this.groupCreator != null && this.groupCreator.equalsIgnoreCase(this.email);
}

public boolean isGroupAdmin(String email) { // With email parameter
    return this.groupCreator != null && this.groupCreator.equalsIgnoreCase(email);
}
```

4. Method Overriding
Definition: Providing specific implementation of methods defined in parent classes.
Implementation Examples:

Gadget Class (`Gadget.java:207-226`)

```
java
@Override
public String toString() {                  // Overriding Object.toString()
    return String.format("%s %s in %s - %s (%.1fW)", type, model, roomName, status,
powerRatingWatts);
}
@Override
public boolean equals(Object obj) {         // Overriding Object.equals()
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    Gadget gadget = (Gadget) obj;
    return type.equals(gadget.type) && roomName.equals(gadget.roomName);
}
@Override
public int hashCode() {                     // Overriding Object.hashCode()
    return (type + roomName).hashCode();
}
```

5. Composition
Definition: "Has-a" relationship where one class contains objects of other classes.
Implementation Examples:

Customer contains Gadgets (`Customer.java:13-17`)

```
java
public class Customer {
```

```
    private List<Gadget> gadgets;                           // Customer HAS gadgets
    private List<DeletedDeviceEnergyRecord> deletedDeviceEnergyRecords;  // Customer HAS records
    private List<DevicePermission> devicePermissions;       // Customer HAS permissions
}
```

SmartHomeService contains multiple services (`SmartHomeService.java:14-23`)
java

```
public class SmartHomeService {
    private final CustomerService customerService;          // Service composition
    private final GadgetService gadgetService;
    private final SessionManager sessionManager;
    private final EnergyManagementService energyService;
    private final TimerService timerService;
    // ... other services
}
```

6. Aggregation
Definition: Weak "has-a" relationship where contained objects can exist independently.
Implementation Examples:

Device Permissions (`DevicePermission.java:6-14`)
java

```
public class DevicePermission {
    private String memberEmail;             // References to external entities
    private String deviceType;              // that exist independently
    private String roomName;
    private String deviceOwnerEmail;
}
```

7. Singleton Design Pattern
Definition: Ensures only one instance of a class exists and provides global access.
Implementation Examples:

SessionManager (`SessionManager.java:3-12`)
java

```
public class SessionManager {
    private static SessionManager instance;  // Static instance

    private SessionManager() {}              // Private constructor

    public static SessionManager getInstance() {  // Static factory method
        if (instance == null) {
            instance = new SessionManager();
        }
        return instance;
    }
}
```

AlertService (`AlertService.java:10-22`)
java

```
public class AlertService {
    private static AlertService instance;

    private AlertService() {                 // Private constructor
        this.userAlerts = new HashMap<>();
    }
    public static synchronized AlertService getInstance() {  // Thread-safe singleton
        if (instance == null) {
            instance = new AlertService();
        }
        return instance;
    }
}
```

8. Nested/Inner Classes
Definition: Classes defined within other classes for logical grouping and encapsulation.
Implementation Examples:

AlertService.Alert (`AlertService.java:24-50`)

```
java
public class AlertService {
    // ...
    public static class Alert {              // Static nested class
        private String alertId;
        private String alertName;
        private String deviceType;
        private AlertType alertType;

        public Alert(String alertId, String alertName, /.../) {
            this.alertId = alertId;
            // ... initialization
        }
    }
}
```

9. Enums
Definition: Special class type that represents a group of constants.
Implementation Examples:

Gadget Enums (`Gadget.java:7-12`)
java

```
public enum GadgetType {
    TV, AC, FAN, ROBO_VAC_MOP
}
public enum GadgetStatus {
    ON, OFF
}
```

AlertService Enum (`AlertService.java:73`)
java

```
public enum AlertType {
    // Alert type constants defined here
}
```

10. Access Modifiers
Definition: Keywords that control the visibility and accessibility of classes, methods, and variables.
Implementation Examples:

- Private: `private String email;` - Only accessible within the same class
- Public: `public String getEmail()` - Accessible from anywhere
- Package-private: Default access for utility methods
- Static: `public static SessionManager getInstance()` - Class-level methods

11. Static Methods and Factory Methods
Definition: Methods that belong to the class rather than instances, often used for utility functions or object creation.
Implementation Examples:

Gadget Factory Method (`Gadget.java:102-123`)
java

```
private static double getDefaultPowerRating(String deviceType) {  // Static utility method
    switch (deviceType.toUpperCase()) {
        case "TV": return 150.0;
        case "AC": return 1500.0;
        case "FAN": return 75.0;
        // ... more cases
```

```
            default: return 50.0;
        }
}
```

12. Data Abstraction
Definition: Hiding implementation details while showing only essential features.
Implementation Examples:

- Service Layer Abstraction: Service classes abstract complex business logic
- Model Classes: Abstract away database storage details
- Utility Classes: Abstract common operations like session management

Architecture Benefits
Layered Architecture
1. Model Layer: Data representation and basic operations
2. Service Layer: Business logic and complex operations
3. Utility Layer: Common functionality and system-level operations

OOP Benefits Realized
- Modularity: Code organized into logical, reusable components
- Maintainability: Clear separation of concerns makes updates easier
- Reusability: Service classes and models can be reused across the application
- Testability: Each component can be tested independently
- Scalability: New features can be added without affecting existing code

Key Classes and Their OOP Features

Class	**Primary OOP Concepts**
`Customer`	Encapsulation, Constructor Overloading, Method Overloading, Composition
`Gadget`	Encapsulation, Enums, Method Overriding, Constructor Overloading
`SmartHomeService`	Composition, Dependency Management
`SessionManager`	Singleton Pattern, Encapsulation
`AlertService`	Singleton Pattern, Nested Classes, Enums
`DevicePermission`	Encapsulation, Method Overriding, Aggregation

File Locations Reference
- Model Classes: `src/main/java/com/smarthome/model/`
  - `Customer.java:1-260`
  - `Gadget.java:1-227`
  - `DevicePermission.java:1-107`
  - `DeletedDeviceEnergyRecord.java`

- Service Classes: `src/main/java/com/smarthome/service/`
  - `SmartHomeService.java:1-50+`
  - `AlertService.java:1-50+`
  - `CustomerService.java`
  - `GadgetService.java`
  - `TimerService.java`
  - `WeatherService.java`

- Utility Classes: `src/main/java/com/smarthome/util/`
  - `SessionManager.java:1-28`
  - `DynamoDBConfig.java`

- Main Application: `src/main/java/com/smarthome/SmartHomeDashboard.java`